

**AD23B31 Image Processing and Computer Vision**

**Image Processing Web Application using Flask  
and OpenCV**

**A PROJECT REPORT**

*Submitted by*

**2116230701073 – DHARANI KUMAR S**



**May2026**

## **BONAFIDE CERTIFICATE**

Certified that this project report titled **“Interactive Real-Time Traffic Light & Color Detection System with GUI”** is the bonafide work carried out by **DHARANI KUMAR S (230701073)** under my supervision and guidance in partial fulfillment of the requirements for the Mini Project in the Department of Computer Science and Engineering during the academic year 2025 – 2026.

**SIGNATURE OF THE FACULTY INCHARGE**

**Submitted for the Practical Examination held on** \_\_\_\_\_

**SIGNATURE OF THE INTERNAL EXAMINER**

# **TABLE OF CONTENTS**

## **1. INTRODUCTION**

- 1.1 Background and Motivation
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Report

## **2. LITERATURE REVIEW**

- 2.1 Traditional Image Processing Methods
- 2.2 OpenCV-Based Systems
- 2.3 Existing Web-Based Image Editors
- 2.4 Gap Identified

## **3. SYSTEM DESCRIPTION**

- 3.1 Project Overview
- 3.2 Features of the System
- 3.3 Technology Stack
- 3.4 System Architecture

## **4. PREPROCESSING AND IMAGE PROCESSING METHODOLOGY**

- 4.1 Webcam Capture and Frame Processing
- 4.2 Traffic Light Detection Module
- 4.3 Multi-Color Detection Module
- 4.4 GUI Control Module
- 4.5 Image Capture Module
- 4.6 Sound Alert Module
- 4.7 Detection Information Display Module
- 4.8 Mode Switching Module

## **5. IMPLEMENTATION DETAILS**

- 5.1 GUI Design
- 5.2 System Implementation
- 5.3 Detection Process
- 5.4 Image Storage and Retrieval
- 5.5 Error Handling

## **6. RESULTS AND ANALYSIS**

6.1 Functional Results

6.2 Detection Performance Analysis

6.3 User Interface Evaluation

## **7. DISCUSSION**

## **8. CONCLUSION AND FUTURE WORK**

## **9. REFERENCES**

## **10. APPENDIX A — SOURCE CODE**

## **11. APPENDIX B — SAMPLE OUTPUT SCREENSHOTS**

## **ABSTRACT**

Computer vision and image processing technologies play an important role in modern intelligent systems by enabling computers to analyze and understand visual information in real time. This project presents an Interactive Real-Time Traffic Light & Color Detection System with GUI developed using Python, OpenCV, and Tkinter. The system is designed to detect traffic signal colors and identify multiple object colors through a webcam-based interface.

The application uses OpenCV as the core computer vision library for image processing and color detection operations. HSV color segmentation and pixel-based color analysis techniques are used to identify traffic signal states such as red, yellow, and green. The system also supports multi-color detection for recognizing common object colors including blue, pink, purple, black, white, and brown. A graphical user interface (GUI) is developed using Tkinter to provide interactive controls such as start, stop, capture, exit, and mode switching. The project includes additional functionalities such as real-time detection display, image capture with automatic labeling, sound alerts during red signal detection, and live status information. The application processes webcam frames continuously and displays the detection results instantly, providing a lightweight and efficient real-time monitoring system. The proposed system demonstrates the practical implementation of classical computer vision techniques without requiring complex machine learning models or training datasets. The project provides a beginner-friendly and cost-effective solution for understanding real-time image processing concepts and interactive GUI-based application development.

Overall, the project serves as an educational and practical demonstration of computer vision applications in traffic monitoring and color recognition systems. Future enhancements may include deep learning-based detection, mobile application integration, cloud connectivity, object tracking, and intelligent automation features for advanced smart monitoring applications.

# 1. INTRODUCTION

## 1.1 Background and Motivation

Computer vision is an important area of image processing that focuses on analyzing and identifying visual information using computational techniques. With the rapid growth of intelligent systems, real-time traffic monitoring and color detection applications have become essential in fields such as transportation, robotics, automation, surveillance, education, and smart systems. Traditional traffic monitoring systems often require expensive hardware, advanced AI models, high computational resources, and technical expertise. GUI-based computer vision applications provide a lightweight and accessible alternative, enabling users to perform real-time traffic signal and color detection directly through a webcam without requiring complex hardware or deep learning models. This project is motivated by the need to develop a simple and beginner-friendly real-time traffic light and color detection system that demonstrates the practical implementation of computer vision techniques using OpenCV and Tkinter.

## 1.2 Problem Statement

Many existing traffic monitoring and computer vision systems are either too complex for beginners or require expensive hardware, advanced AI models, and high computational resources. There is a need for a lightweight real-time application that allows users to perform traffic light and color detection operations in a simple, interactive, and user-friendly environment using basic computer vision techniques.

## 1.3 Objectives

The objectives of this project are:

- To develop a real-time traffic light and color detection system using Python, OpenCV, and Tkinter.
- To implement traffic signal detection and multi-color recognition techniques using computer vision methods.
- To provide real-time webcam-based detection and monitoring functionality.
- To enable users to capture, detect, and label images easily through an interactive GUI.
- To design a lightweight and beginner-friendly graphical user interface for real-time computer vision applications.

## 1.4 Scope of the Report

This report focuses on the design and implementation of a real-time traffic light and color detection system using Python, OpenCV, and Tkinter. The system supports traffic signal detection, multi-color recognition, image capture, sound alerts, and GUI-based interaction. The report includes system architecture, implementation details, computer vision techniques, and real-time detection functionality. Advanced deep learning models, cloud integration, and AI-based object tracking are beyond the scope of this project.

## **2. LITERATURE REVIEW**

Computer vision and real-time detection systems have become widely used in modern intelligent applications. Researchers and developers have proposed various approaches for implementing traffic monitoring and color recognition systems using image processing techniques.

### **2.1 Traditional Image Processing Methods**

Traditional computer vision methods involve analyzing pixel values and color information to detect and recognize objects or patterns in images and videos. Common techniques include color segmentation, thresholding, contour detection, and HSV color space analysis.

### **2.2 OpenCV-Based Systems**

OpenCV is one of the most popular open-source libraries used for computer vision and image processing tasks. It provides optimized implementations for color detection, object tracking, image filtering, feature extraction, and real-time video processing.

### **2.3 Existing Web-Based Image Editors**

Several intelligent traffic monitoring and object recognition systems are available in modern smart transportation and automation environments. These systems support traffic signal recognition, object detection, surveillance, and automated monitoring functionalities. However, many existing systems require expensive hardware, deep learning models, large datasets, and high computational resources. Educational computer vision projects mainly focus on providing simpler and lightweight alternatives for learning and experimentation purposes.

### **2.4 Gap Identified**

Although many intelligent monitoring systems are available, beginner-friendly applications demonstrating the integration of OpenCV and Tkinter for real-time traffic light and color detection are limited. Many existing systems also lack transparency in explaining how computer vision algorithms work internally.

This project addresses these gaps by developing a simple and understandable real-time traffic light and color detection system with GUI-based interaction, clearly implemented detection techniques, and live webcam processing functionality.

### 3. SYSTEM DESCRIPTION

#### 3.1 Project Overview

The Interactive Real-Time Traffic Light & Color Detection System with GUI is a webcam-based computer vision application that allows users to detect traffic signal colors and multiple object colors in real time using OpenCV. The application processes video frames continuously and displays the detection results instantly through a graphical user interface developed using Tkinter.

The system supports traffic signal detection, multi-color recognition, image capture with automatic labeling, sound alerts, real-time detection display, and mode switching functionality.

#### 3.2 Features of the System

The main features of the system include:

- Real-time Webcam Detection
- Traffic Light Detection
- Multi-Color Detection
- HSV Color Segmentation
- GUI-Based User Interface
- Start and Stop Camera Controls
- Image Capture with Automatic Labeling
- Sound Alert for Red Signal Detection
- Traffic and Multi-Color Mode Switching
- Live Detection Information Display
- Real-Time Video Processing
- Lightweight and Beginner-Friendly System

#### 3.3 Technology Stack

The following technologies are used in this project:

| Component             | Technology Used |
|-----------------------|-----------------|
| Programming Language  | Python          |
| GUI Framework         | Tkinter         |
| Computer Vision       | OpenCV          |
| Numerical Computation | NumPy           |
| Image Handling        | Pillow (PIL)    |



### **3.4 System Architecture**

The system architecture consists of three major components:

#### **1. GUI Interface**

The GUI provides an interactive user interface where users can start the camera, stop detection, switch modes, capture images, and view real-time detection results.

#### **2. Camera and Processing Module**

The webcam captures live video frames continuously and sends them for processing and analysis.

#### **3. OpenCV Detection Engine**

OpenCV performs traffic signal detection, multi-color recognition, HSV color segmentation, and image processing operations in real time.

The workflow of the system is as follows:

1. User starts webcam detection
2. Webcam captures video frames
3. OpenCV processes the frames
4. System detects traffic signals or object colors
5. Detection results are displayed on GUI
6. User captures and stores detected images
7. Sound alerts are generated during red signal detection

### **4. MODULES DESCRIPTION**

#### **4.1 Webcam Capture and Frame Processing**

The system captures live video continuously using a webcam. The captured video frames are resized and processed in real time for traffic signal and color detection operations. The webcam module provides live input to the computer vision processing engine and enables continuous monitoring functionality.

#### **4.2 Traffic Light Detection Module**

The traffic light detection module identifies red, yellow, and green traffic signals using HSV color segmentation techniques.

The process includes:

- Converting video frames from BGR to HSV color space
- Applying color threshold masks
- Counting dominant color pixels
- Identifying traffic signal state

The module displays:

- STOP for red signal
- WAIT for yellow signal
- GO for green signal

This module helps simulate basic intelligent traffic monitoring functionality.

### **4.3 Multi-Color Detection Module**

The multi-color detection module identifies different object colors using pixel-based color analysis techniques.

The process includes:

- Extracting the Region of Interest (ROI)
- Calculating average color intensity values
- Comparing RGB values with predefined conditions
- Identifying dominant object color

The module supports detection of colors such as:

- Red
- Green
- Blue
- Yellow
- Orange
- Pink
- Purple
- Black
- White
- Brown

This module demonstrates real-time object color recognition using computer vision techniques.

#### **4.4 GUI Control Module**

The GUI control module provides an interactive interface for users to control the system easily. The graphical interface is developed using Tkinter.

#### **4.5 Image Capture Module**

The image capture module allows users to save detected frames directly from the webcam. Captured images are automatically labeled based on the detected color or traffic signal.

#### **4.6 Sound Alert Module**

The sound alert module generates an audio beep whenever a red traffic signal is detected. This module helps simulate intelligent traffic warning systems and improves real-time alert functionality.

#### **4.7 Detection Information Display Module**

This module displays real-time system information directly on the video frame. This module improves monitoring and helps users understand the system output clearly.

#### **4.8 Mode Switching Module**

The mode switching module allows users to switch between:

- Traffic Detection Mode
- Multi-Color Detection Mode

Traffic mode focuses on detecting traffic signals such as red, yellow, and green.

Multi-color mode identifies common object colors using pixel-based color analysis techniques.

The module increases system flexibility and enables multiple functionalities within a single application.

## **5. IMPLEMENTATION AND RESULTS**

### **5.1 GUI Design**

The graphical user interface is developed using Tkinter. The GUI provides an interactive and user-friendly environment for controlling the detection system.

Key GUI components include:

- Webcam display window
- Start button
- Stop button
- Capture button
- Exit button
- Mode switch button
- Real-time detection display

The GUI enables smooth real-time interaction without requiring command-line operations.

### **5.2 System Implementation**

The system is implemented using Python and OpenCV. The implementation performs the following tasks:

- Captures webcam video frames
- Processes frames in real time
- Detects traffic signals and object colors
- Displays detection results on GUI
- Generates sound alerts
- Captures and stores images automatically
- Switches between traffic and multi-color detection modes

### **5.3 Detection Process**

The application contains the following detection processes:

ProcessFunctionWebcam CaptureCaptures live video  
framesHSV ConversionConverts frames into HSV  
color spaceTraffic Signal DetectionDetects red,  
yellow, and green signalsMulti-Color DetectionDetects  
multiple object colorsGUI DisplayDisplays real-time  
resultsSound AlertGenerates beep during red  
detectionImage CaptureSaves detected frames  
automatically.

## 5.4 Image Storage and Retrieval

Captured images are stored automatically in the project directory with labels based on detected colors.

Examples:

- RED\_0.jpg
- GREEN\_1.jpg
- BLUE\_2.jpg

The system dynamically captures and stores processed frames for future reference and analysis.

## 5.5 Error Handling

The system includes error handling mechanisms for:

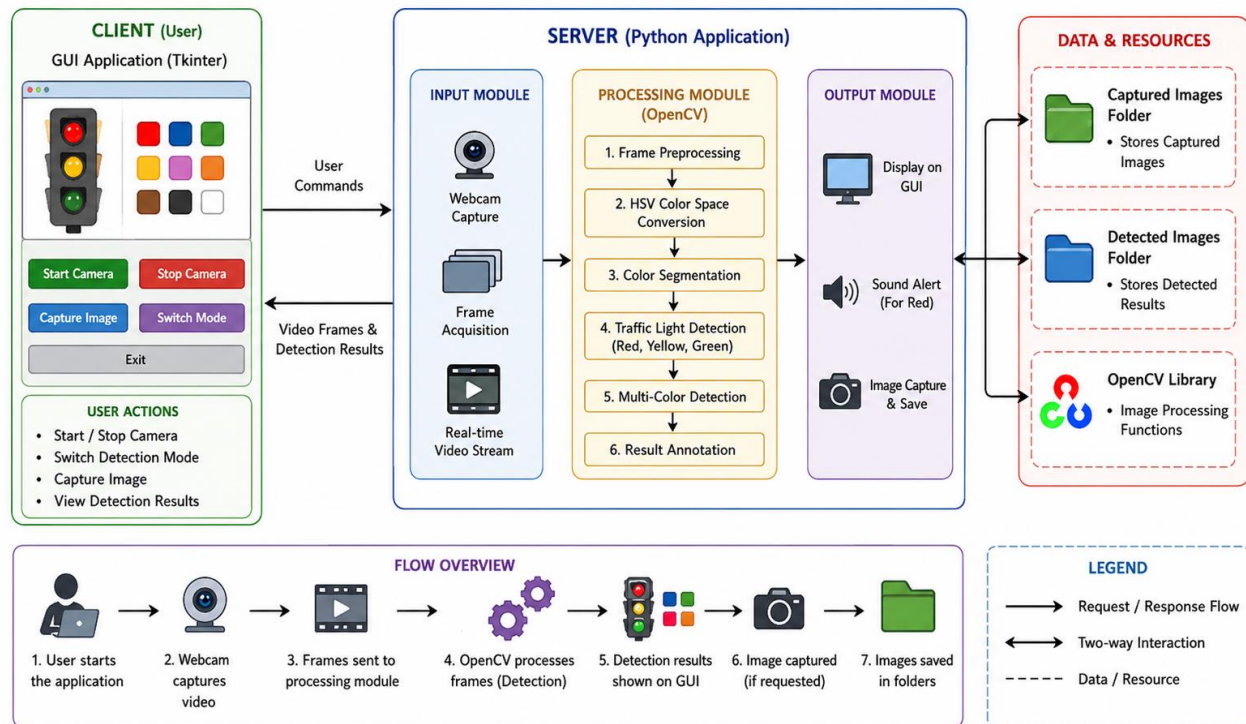
- Webcam access failure
- Invalid frame capture
- Empty frame processing
- Detection errors
- Camera initialization issues

These mechanisms improve system reliability and provide stable real-time performance.

**Table 3: Implementation Configuration**

| Parameter                | Value                                                                 |
|--------------------------|-----------------------------------------------------------------------|
| Project                  | Interactive Real-Time Traffic Light & Color Detection System with GUI |
| Programming Language     | Python 3.10                                                           |
| GUI Framework            | Tkinter                                                               |
| Computer Vision Library  | OpenCV (cv2)                                                          |
| Numerical Computation    | NumPy                                                                 |
| Image Handling           | Pillow (PIL)                                                          |
| Alert System             | Winsound                                                              |
| Input Source             | Webcam (Live Video Stream)                                            |
| Detection Modes          | Traffic Light Detection, Multi-Color Detection                        |
| Detected Traffic Signals | Red, Yellow, Green                                                    |
| Detected Colors          | Red, Green, Blue, Yellow, Orange, Pink, Purple, Black, White, Brown   |
| Image Capture Format     | JPG                                                                   |
| Real-time Processing     | Yes                                                                   |
| Sound Alert              | Beep on Red Signal Detection                                          |
| Development Environment  | Visual Studio Code                                                    |
| Operating System         | Windows 10 / 11                                                       |
| Hardware Requirements    | Min. 4GB RAM, Dual-Core Processor, Integrated Webcam                  |
| Additional Features      | Real-time Detection, Mode Switching, Image Capture, GUI Display       |
| Libraries / Modules Used | OpenCV, NumPy, Tkinter, Pillow, Winsound                              |

## ARCHITECTURE DIAGRAM – TRAFFIC LIGHT & COLOR DETECTION SYSTEM WITH GUI



## 6. RESULTS AND ANALYSIS

### 6.1 Functional Results

The developed application successfully performs all implemented traffic light and color detection operations. Users can:

- Start and stop webcam detection successfully
- Detect traffic signals in real time
- Identify multiple object colors
- Switch between traffic and multi-color modes
- Capture and store detected images
- Receive sound alerts during red signal detection

The system demonstrates smooth interaction between the GUI interface and the OpenCV processing engine.

6.2 Detection Performance Analysis

Table 1 presents the implemented modules and their functionality.

Table 1: Detection Modules

|                       |                          |                   |                                       |
|-----------------------|--------------------------|-------------------|---------------------------------------|
| Module                | Purpose                  | Traffic Detection | Detect red, yellow, and green signals |
| Multi-Color Detection | Detect object colors     | HSV Segmentation  | Separate colors using HSV ranges      |
| GUI                   | Provide user interaction | Sound Alert       | Generate alert for red signal         |
| Image Capture         | Save detected frames     | Mode Switching    | Switch between detection modes        |

The system produced visually accurate detection results under proper lighting conditions and simple backgrounds.

6.3 User Interface Evaluation

The graphical user interface is designed to be simple, interactive, and beginner-friendly. The real-time display and detection information improve usability by allowing users to monitor detection results instantly. The GUI buttons simplify camera control, image capture, and mode switching operations. The application also works effectively with live webcam input and provides smooth real-time interaction.

7. DISCUSSION

The Interactive Real-Time Traffic Light & Color Detection System with GUI demonstrates the successful integration of OpenCV and Tkinter for implementing real-time computer vision operations in a desktop application environment. One of the major strengths of the system is its simplicity. Users can perform traffic signal and color detection operations without requiring expensive hardware or complex AI models. The lightweight design also makes the application suitable for educational demonstrations and beginner-level computer vision projects. The use of OpenCV provides efficient implementations for real-time image processing and color segmentation techniques. The Tkinter framework simplifies GUI development and user interaction management. The application successfully processes webcam frames with low latency and provides immediate visual feedback. Features such as real-time detection display, sound alerts, image capture, and mode switching improve the overall user experience. However, the project has certain limitations. Detection accuracy may reduce under poor lighting conditions or complex backgrounds. Similar colors may overlap during multi-color detection. The system also uses rule-based computer vision techniques instead of advanced deep learning models. Despite these limitations, the project effectively demonstrates practical applications of computer vision, real-time image processing, and GUI-based system development technologies.



## 8. CONCLUSION AND FUTURE WORK

This mini project report presented the design and implementation of an Interactive Real-Time Traffic Light & Color Detection System with GUI using Python, OpenCV, and Tkinter.

The application successfully performs multiple real-time computer vision operations including traffic signal detection, multi-color recognition, image capture, sound alert generation, and GUI-based interaction.

The project demonstrates how computer vision techniques can be integrated with real-time webcam processing and graphical user interface technologies to create interactive and beginner-friendly applications.

The key contributions of this project are:

- Development of a complete real-time traffic light and color detection system using OpenCV and Tkinter.
- Implementation of traffic signal detection and multi-color recognition techniques.
- Integration of webcam processing with GUI-based interaction.
- Interactive and beginner-friendly user interface design.
- Real-time detection, image capture, and alert generation functionality.

Future work directions include:

- Adding deep learning-based object and color detection techniques.
- Supporting automatic traffic signal classification using AI models.
- Integrating cloud-based monitoring and storage systems.
- Developing mobile and IoT-based monitoring applications.
- Implementing object tracking and intelligent automation features.

## REFERENCES

- [1] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.
- [2] A. Kaehler and G. Bradski, Learning OpenCV: Computer Vision with the OpenCV Library, O’Reilly Media, 2008.
- [3] M. Nixon and A. Aguado, Feature Extraction and Image Processing for Computer Vision, Academic Press, 2019.
- [4] Python Documentation, “Python Programming Language,” [Online]. Available: Python Official Documentation
- [5] OpenCV Documentation, “OpenCV Library,” [Online]. Available: OpenCV Documentation
- [6] NumPy Documentation, “NumPy Library,” [Online]. Available: NumPy Documentation
- [7] Tkinter Documentation, “Tkinter GUI Library,” [Online]. Available: Tkinter Documentation
- [8] R. Szeliski, Computer Vision: Algorithms and Applications, Springer, 2011.
- [9] R. Gonzalez and R. Woods, Digital Image Processing, Pearson Education, 2018.

## APPENDIX A — SOURCE CODE

```
import cv2
import numpy as np
import tkinter as tk
from PIL import Image, ImageTk
import winsound

# Initialize camera
cap = cv2.VideoCapture(0)
running = False
current_frame = None

# Variables
mode = "TRAFFIC"
last_detected = "NONE"
image_count = 0

def start_camera():
    global running
    running = True
    update_frame()

def stop_camera():
    global running
    running = False

def exit_app():
    cap.release()
    root.destroy()
```

```

def capture_image():
    global image_count, last_detected
    if current_frame is not None:
        filename = f'{last_detected}_{image_count}.jpg'
        cv2.imwrite(filename, current_frame)
        print(f'Saved: {filename}')
        image_count += 1

def toggle_mode():
    global mode
    mode = "MULTI" if mode == "TRAFFIC" else "TRAFFIC"
    print("Mode:", mode)

def detect(frame):
    global last_detected

    frame = cv2.resize(frame, (640, 480))
    roi = frame[100:400, 150:500]

    center = roi[100:200, 150:250]

    hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

    detected = "NONE"

    # Traffic Detection Mode
    if mode == "TRAFFIC":

        lower_red1 = np.array([0, 120, 70])
        upper_red1 = np.array([10, 255, 255])

        lower_red2 = np.array([170, 120, 70])
        upper_red2 = np.array([180, 255, 255])

        lower_green = np.array([40, 70, 70])
        upper_green = np.array([90, 255, 255])

        lower_yellow = np.array([18, 100, 100])
        upper_yellow = np.array([35, 255, 255])

        mask_red = cv2.inRange(hsv, lower_red1, upper_red1) + \
            cv2.inRange(hsv, lower_red2, upper_red2)

        mask_green = cv2.inRange(hsv, lower_green, upper_green)

        mask_yellow = cv2.inRange(hsv, lower_yellow, upper_yellow)

        r = cv2.countNonZero(mask_red)
        g = cv2.countNonZero(mask_green)
        y = cv2.countNonZero(mask_yellow)

        max_val = max(r, g, y)

```

```

if max_val > 3000:

    if max_val == r:
        detected = "RED"

        cv2.putText(frame, "STOP", (20, 50),
                     cv2.FONT_HERSHEY_SIMPLEX,
                     1, (0, 0, 255), 3)

        if last_detected != "RED":
            winsound.Beep(1000, 200)

    elif max_val == y:
        detected = "YELLOW"

        cv2.putText(frame, "WAIT", (20, 50),
                     cv2.FONT_HERSHEY_SIMPLEX,
                     1, (0, 255, 255), 3)

    elif max_val == g:
        detected = "GREEN"

        cv2.putText(frame, "GO", (20, 50),
                     cv2.FONT_HERSHEY_SIMPLEX,
                     1, (0, 255, 0), 3)

```

# Multi-Color Detection Mode  
else:

```

avg_color = cv2.mean(center)[:3]
b, g, r = avg_color

if r > 150 and g < 80 and b < 80:
    detected = "RED"

elif g > 150 and r < 80 and b < 80:
    detected = "GREEN"

elif b > 150 and r < 80 and g < 80:
    detected = "BLUE"

elif r > 150 and g > 150 and b < 80:
    detected = "YELLOW"

elif r > 150 and g > 80 and b < 50:
    detected = "ORANGE"

elif r > 150 and b > 150:
    detected = "PINK"

elif b > 120 and r > 120:

```

```

        detected = "PURPLE"

    elif r < 50 and g < 50 and b < 50:
        detected = "BLACK"

    elif r > 200 and g > 200 and b > 200:
        detected = "WHITE"

    elif 80 < r < 150 and 40 < g < 100:
        detected = "BROWN"

    else:
        detected = "UNKNOWN"

    cv2.putText(frame, detected, (20, 50),
                cv2.FONT_HERSHEY_SIMPLEX,
                1, (255, 255, 255), 3)

    last_detected = detected

    cv2.putText(frame, f"Mode: {mode}", (20, 420),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6, (255, 255, 255), 2)

    cv2.putText(frame, f"Detected: {detected}", (20, 450),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6, (255, 255, 255), 2)

    cv2.rectangle(frame, (150, 100),
                  (500, 400), (255, 255, 255), 2)

    return frame

def update_frame():
    global current_frame

    if running:
        ret, frame = cap.read()

        if ret:
            frame = detect(frame)
            current_frame = frame.copy()

            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            img = Image.fromarray(rgb)
            imgtk = ImageTk.PhotoImage(image=img)

            label.imgtk = imgtk
            label.configure(image=imgtk)

    root.after(10, update_frame)

```

```
# GUI Setup
root = tk.Tk()
root.title("Traffic Light Detection GUI")

label = tk.Label(root)
label.pack()

btn_frame = tk.Frame(root)
btn_frame.pack()

tk.Button(btn_frame, text="Start",
          command=start_camera,
          width=10).grid(row=0, column=0)

tk.Button(btn_frame, text="Stop",
          command=stop_camera,
          width=10).grid(row=0, column=1)

tk.Button(btn_frame, text="Capture",
          command=capture_image,
          width=10).grid(row=0, column=2)

tk.Button(btn_frame, text="Exit",
          command=exit_app,
          width=10).grid(row=0, column=3)

tk.Button(btn_frame, text="Mode",
          command=toggle_mode,
          width=10).grid(row=0, column=4)

root.mainloop()
```

## APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

